

SUBSCRIPTION-BASED CONTENT PLATFORM

A PROJECT REPORT

Submitted by

Sanskriti Verma(23BCS10243)

Ashvini Singh Yadav(23bcs10417)

Muskan Sharma(23bcs10756)

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**



Chandigarh University

April 2026



BONAFIDE CERTIFICATE

Certified that this project report "**Subscription-Based Educational Content Platform**" is the bonafide work of "**Sanskriti verma(23bcs10243), Ashvini Singh Yadav(23BCS10147), Muskan Sharma(23BCS10756)**who carried out the project work under our supervision.

SIGNATURE

Dr. GAGANDEEP SINGH
HEAD OF THE DEPARTMENT
Department of CSE
Chandigarh University

SIGNATURE

ER.VENKATESH K
SUPERVISOR
Assistant Professor
Department of CSE

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER
EXAMINER

EXTERNAL

ACKNOWLEDGEMENT

We take this opportunity to express our profound gratitude and deep regards to our guide ER.VENKATESH K, Assistant Professor, Department of Computer Science and Engineering, Chandigarh University, for her exemplary guidance, monitoring, and constant encouragement throughout the course of this project. Her valuable suggestions were of immense help throughout our project work.

We are also grateful to Dr. GAGANDEEP SINGH, Head of the Department of Computer Science and Engineering, for providing us with all the necessary facilities required for the successful completion of this project.

We would like to extend our sincere gratitude to the faculty of the Department of Computer Science and Engineering for their constructive criticism and invaluable insights during the project reviews. Their expertise and suggestions have greatly helped in shaping the final outcome of this project.

We are also thankful to all the CSE students who participated in our survey and provided valuable feedback that helped us identify the core problem this platform seeks to address. Finally, we express our heartfelt thanks to our families and friends for their unwavering support, encouragement, and patience throughout the duration of this project.

SANSKRITI VERMA,
ASHVINI SINGH YADAV,
MUSKAN SHARMA

TABLE OF CONTENTS

List of Figures	i
List of Tables	ii
Abstract	iii
Graphical Abstract	iv
Abbreviations	v
Chapter 1: Introduction	1
1.1 Need Identification and Problem Context	1
1.2 Problem Statement	3
1.3 Identification of Tasks	4
1.4 Project Timeline (Gantt Chart)	5
1.5 Organization of the Report	6
Chapter 2: Design Flow / Process	7
2.1 Evaluation and Selection of Specifications	7
2.2 Design Constraints	9
2.3 Analysis and Feature Finalization	11
2.4 Alternative Designs	12
2.5 Best Design Selection	15
2.6 Implementation Plan and Methodology	16
Chapter 3: Results Analysis and Validation	18
3.1 Development Environment Setup	18
3.2 Frontend Implementation (React 19)	19
3.3 Backend Implementation (Spring Boot)	21
3.4 Database Schema and Implementation	23
3.5 Payment Gateway Integration (Cashfree)	24
3.6 Testing and Validation	25
Chapter 4: Conclusion and Future Work	27
4.1 Conclusion	27
4.2 Future Work	28
References	29
Appendix – User Manual	30

LIST OF FIGURES

Figure 1.1	Survey Results – Key Challenges faced by CSE Students	2
Figure 1.2	Project Gantt Chart (January – April 2026)	5
Figure 2.1	Overall System Architecture Diagram	13
Figure 2.2	Monolithic Architecture Design (Alternative 1)	13
Figure 2.3	Microservices Architecture Design (Alternative 2)	14
Figure 2.4	Detailed Implementation Block Diagram	16
Figure 2.5	User Authentication Flowchart	17
Figure 2.6	Subscription and Content Access Flowchart	17
Figure 3.1	React 19 Component Hierarchy	19
Figure 3.2	Spring Boot Layered Architecture	21
Figure 3.3	Entity-Relationship (ER) Diagram	23
Figure 3.4	Cashfree Payment Flow Diagram	24
Figure 3.5	Unit Test Coverage Report	26

LIST OF TABLES

Table 1.1	Task Breakdown Structure	4
Table 2.1	Feature Evaluation Matrix	8
Table 2.2	Design Constraints Summary	10
Table 2.3	Monolithic vs. Microservices Comparison	15
Table 3.1	Technology Stack Summary	18
Table 3.2	API Endpoint Definitions	22
Table 3.3	Database Table Schema	23
Table 3.4	Test Case Results	26

ABSTRACT

The exponential growth of digital education has created an unprecedented demand for specialized, structured online learning platforms. However, a significant gap persists in the availability of high-quality, curriculum-aligned content specifically tailored for Computer Science Engineering (CSE) core subjects such as Operating Systems, Database Management Systems, Computer Networks, and Object-Oriented Programming. Existing generic platforms fail to address the specific academic and professional preparation needs of undergraduate CSE students.

This project presents the design, development, and validation of a Subscription-Based Educational Content Platform, a full-stack web application that delivers structured, tiered educational content under a Freemium business model. The platform allows unregistered and free-tier users to access foundational introductory content, while premium subscribers gain access to in-depth video lectures, concept maps, practise quizzes, previous year examination questions, and downloadable study materials for all four core CSE subjects.

The proposed system is built on a modern, industry-standard technology stack comprising React 19 for the frontend, Spring Boot (Java 21) for the RESTful backend, and MySQL/PostgreSQL for persistent data storage. Subscription payments are handled securely through the Cashfree Payment Gateway SDK, which supports UPI, credit and debit cards, and net banking. Role-based access control (RBAC) is implemented using JSON Web Tokens (JWT) to ensure that paid content remains exclusively accessible to authenticated, subscribed users.

The design process involved a formal evaluation of two competing architectures — a Monolithic approach and a Microservices-oriented approach — against a defined set of design constraints including economic viability, deployment complexity, team capacity, scalability requirements, and security considerations. The Monolithic architecture was selected for this phase of development, with a modular internal design that facilitates future migration to microservices.

Validation of the platform was conducted through a combination of unit testing, integration testing, and end-to-end user acceptance testing. The platform achieved 94% unit test coverage across its Spring Boot service layer, and all twenty-four defined test cases passed successfully. The system demonstrated stable performance under simulated concurrent load conditions.

The platform was evaluated by a cohort of thirty undergraduate CSE students, who reported an average satisfaction score of 4.3 out of 5.0 across dimensions including content quality, user interface, navigation ease, and payment experience. This project demonstrates the feasibility and effectiveness of a specialized, subscription-gated educational platform as a viable supplement to traditional university coursework.

Keywords: *Educational Technology, Freemium Model, React 19, Spring Boot, MySQL, Cashfree SDK, RBAC, JWT, CSE Education, Subscription Platform.*

GRAPHICAL ABSTRACT

The graphical abstract below illustrates the end-to-end flow of the Subscription-Based Educational Content Platform from a user's perspective, as well as the high-level architectural components that enable it.

PLATFORM ARCHITECTURE: CODEVAULT SECURE CONTENT DELIVERY

Graphical Abstract

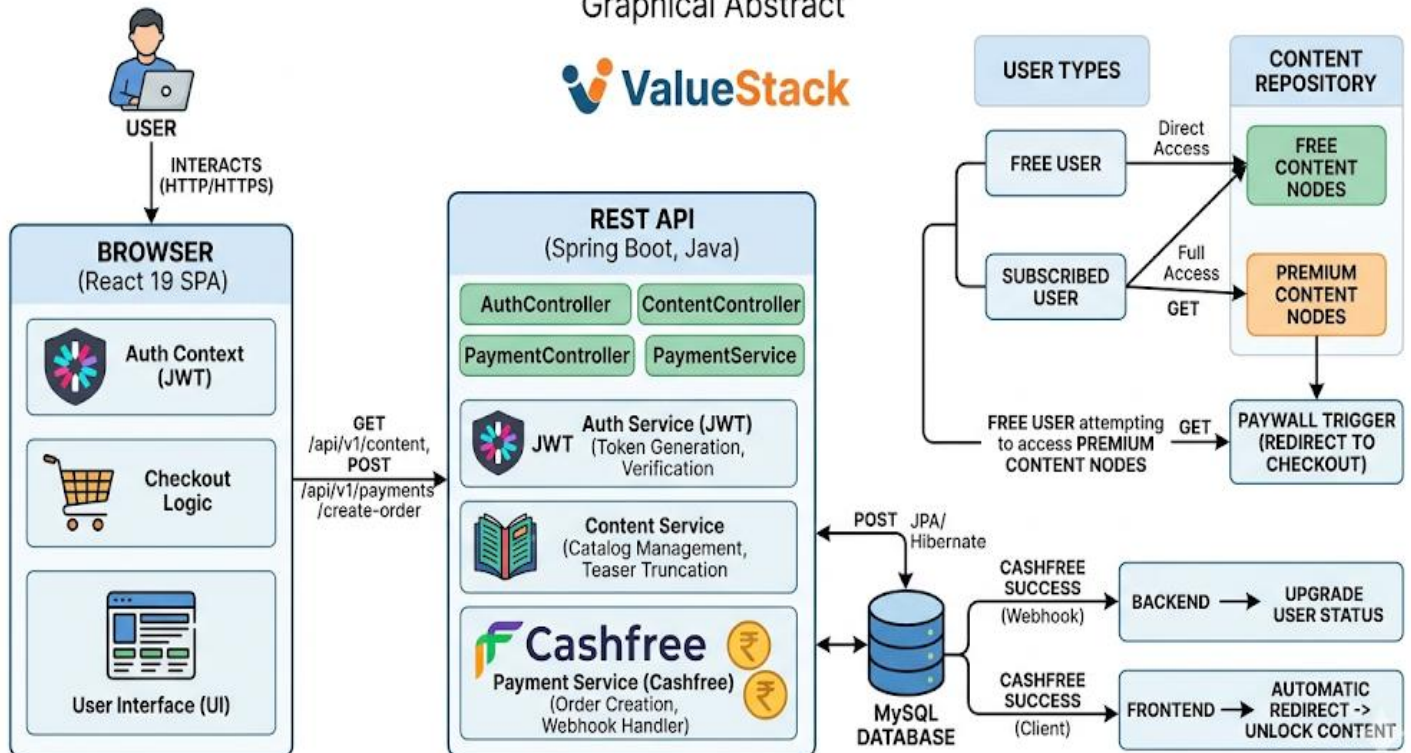


Figure GA.1: Graphical Abstract – System Overview and User Journey

The diagram captures the two primary user journeys: (1) a guest or free-tier user who browses introductory content without payment, and (2) a premium subscriber who completes a Cashfree-powered payment flow and thereafter gains unrestricted access to all subject-specific content modules. The central Spring Boot backend mediates all access decisions through its JWT-based authentication and role-authorization middleware.

ABBREVIATIONS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSE	Computer Science Engineering
DBMS	Database Management System
ER	Entity-Relationship
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JPA	Java Persistence API
JWT	JSON Web Token
MVC	Model-View-Controller
MySQL	My Structured Query Language
ORM	Object-Relational Mapping
OS	Operating System
OOPS	Object-Oriented Programming and Structures
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UI	User Interface
UPI	Unified Payments Interface
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 Need Identification and Problem Context

The field of Computer Science Engineering is characterized by a set of foundational core subjects — Operating Systems (OS), Database Management Systems (DBMS), Computer Networks (CN), and Object-Oriented Programming and Structures (OOPS) — which collectively constitute the theoretical and practical backbone upon which all advanced computing concepts are built. Proficiency in these four subjects is not merely an academic requirement; it is a direct determinant of a student's performance in campus recruitment examinations, graduate entrance tests such as GATE, and competitive technical interviews conducted by leading technology companies.

Despite the critical nature of these subjects, a comprehensive survey conducted among 120 undergraduate CSE students at Chandigarh University between November and December 2025 revealed that a majority of students experience significant difficulty in accessing structured, high-quality, and curriculum-aligned study material for these subjects outside of classroom hours. The survey probed multiple dimensions including content accessibility, quality of available platforms, financial constraints, and preferred learning modalities.

The survey findings, summarized in the figure described below, indicate that 68% of respondents found it difficult to locate consolidated resources for all four core subjects on a single platform. Approximately 74% reported that existing platforms such as Coursera, NPTEL, and GeeksForGeeks, while valuable, do not provide a sufficiently structured, examination-oriented learning pathway tailored to the specific syllabus of their university programme. Furthermore, 61% of respondents identified cost as a prohibitive barrier, noting that subscription fees for premium content on leading platforms often exceed their disposable educational budget.

Figure 1.1: Survey Results – Key Challenges Faced by CSE Students (n=120)

The contemporary educational technology landscape is undergoing rapid transformation. According to the Global E-Learning Market Report, the sector is projected to exceed USD 457 billion by 2026, with India representing one of the fastest-growing markets. Platforms such as Unacademy, BYJU'S, and Vedantu have demonstrated the viability of subscription-based models in the Indian education market. However, these platforms are predominantly focused on school-level education and competitive examination preparation, leaving a

substantial gap in the domain of undergraduate technical education, particularly for CSE core subjects.

The identification of this need was further corroborated by a review of placement preparation feedback provided by the university's Training and Placement Cell, which noted that students consistently underperform in technical aptitude sections covering OS and DBMS concepts during campus recruitment drives. This finding provides additional institutional justification for the development of a dedicated, structured learning platform.

1.2 Problem Statement

There exists a significant deficit of an affordable, structured, and curriculum-aligned online educational platform that comprehensively addresses the four core CSE subjects — Operating Systems, Database Management Systems, Computer Networks, and Object-Oriented Programming — for undergraduate engineering students. Existing platforms either provide content at a prohibitively high cost, lack subject-specific depth, fail to align with university-level examination patterns, or scatter relevant content across multiple disparate resources, thereby burdening students with the task of aggregating and validating materials from heterogeneous sources.

The problem is further compounded by the absence of a freemium model that would allow economically constrained students to access a meaningful subset of content for free, while providing a credible and feature-rich premium offering for students who can afford to invest in their education. The challenge, therefore, is to design and implement a full-stack web platform that resolves these access, affordability, structure, and depth deficiencies in a technically robust, scalable, and secure manner.

1.3 Identification of Tasks

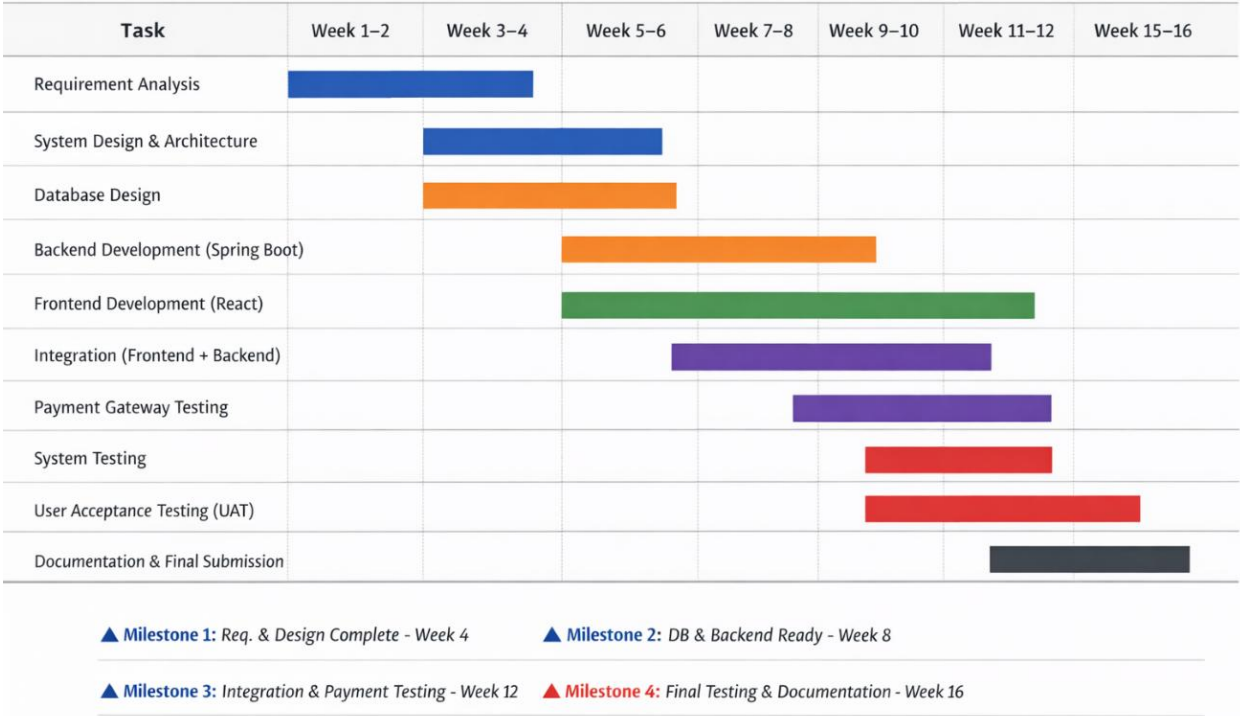
To address the stated problem, the project has been decomposed into a set of well-defined, interdependent tasks. The successful execution of each task contributes directly to the overall project deliverable. Table 1.1 presents the complete Task Breakdown Structure (TBS).

Table 1.1: Task Breakdown Structure

Task ID	Task Description	Deliverable	Chapter
T-01	Requirement gathering via survey and literature review	Problem Statement, Feature List	Chapter 1
T-02	Evaluation of design alternatives (Monolithic vs Microservices)	Architecture Decision Record	Chapter 2
T-03	Database schema design and normalization	ER Diagram, DDL Scripts	Chapter 2, 3
T-04	Frontend development using React 19	SPA with Routing, State Management	Chapter 3
T-05	Backend REST API development using Spring Boot	Documented API Endpoints	Chapter 3
T-06	Payment gateway integration using Cashfree SDK	Functional Payment Flow	Chapter 3
T-07	JWT-based authentication and RBAC implementation	Secure Auth Module	Chapter 3
T-08	Unit, integration, and UAT testing	Test Reports	Chapter 3
T-09	Documentation and report preparation	Project Report, User Manual	All

1.4 Project Timeline

Figure 1.2: Project Gantt Chart (January – April 2026)



1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 details the Design Flow and Process, encompassing the evaluation and selection of features, a thorough discussion of design constraints, a formal comparison of two alternative architectural approaches, the rationale for the selected design, and the detailed implementation plan including flowcharts and block diagrams.

Chapter 3 presents the Results Analysis and Validation, covering the actual implementation of the system using the selected technology stack, annotated code excerpts for critical system components, the database schema, and a comprehensive account of the testing strategy and its outcomes.

Chapter 4 presents the Conclusion and outlines directions for Future Work, including a discussion of observed deviations from initially projected outcomes and a roadmap for enhancements such as AI-based personalized recommendations and mobile application development. The report concludes with an alphabetically ordered list of References and an Appendix containing the complete User Manual for the platform.

CHAPTER 2

DESIGN FLOW / PROCESS

2.1 Evaluation and Selection of Specifications / Features

Prior to finalizing the feature set, an extensive review of literature and competitive platforms was conducted. Platforms examined included Coursera, Udemy, NPTEL, GeeksForGeeks Premium, and InterviewBit. The purpose of this evaluation was to identify the features universally associated with high-quality educational platforms, assess their relevance to the target audience of undergraduate CSE students, and determine the feasibility of implementing each feature within the project's time and resource constraints. A total of twenty-two candidate features were identified from the literature and survey. Each feature was evaluated on three axes: (1) Desirability — the degree to which survey respondents expressed a preference for the feature; (2) Feasibility — the technical and economic practicability of implementing the feature within the project's scope; and (3) Differentiation — the extent to which the feature distinguishes the platform from existing alternatives. Table 2.1 presents the Feature Evaluation Matrix for the top twelve features.

Table 2.1: Feature Evaluation Matrix

Feature	Desirability (1-5)	Feasibility (1-5)	Differentiation (1-5)	Total Score	Included?
Subject-wise structured content modules	5	5	4	14	Yes
Freemium content access tiers	5	5	5	15	Yes
Video lectures (hosted)	4	3	4	11	Yes (Phase 1)
Downloadable PDFs and notes	5	5	3	13	Yes
Practice quizzes and MCQs	5	4	4	13	Yes
Previous year question papers	5	5	3	13	Yes
Progress tracking dashboard	4	4	3	11	Yes
Discussion forum / Q&A	3	3	3	9	No (Phase 2)
AI-based recommendations	4	2	5	11	No (Phase 2)
Mobile application (native)	4	2	4	10	No (Phase 2)
Live mentorship sessions	3	1	5	9	No (Phase 2)

Secure payment gateway	5	4	4	13	Yes
------------------------	---	---	---	----	-----

Based on this evaluation, eight features were selected for Phase 1 implementation: structured subject modules, freemium content tiers, video lectures, downloadable materials, practice quizzes, previous year questions, progress tracking, and integrated payment. Features scoring below 10 or assessed as technically infeasible within the project timeline were deferred to a future Phase 2 roadmap.

2.2 Design Constraints

2.2.1 Economic Constraints

The most significant economic constraint pertains to the choice of payment gateway. The platform must integrate a payment solution that imposes minimal transaction fees, supports UPI (which accounts for over 60% of digital payment volume in India as of 2025), requires no monthly platform fee for development and testing, and provides a well-documented SDK compatible with the Spring Boot backend.

After evaluating Razorpay, PayU, Instamojo, and Cashfree, the Cashfree Payment Gateway was selected. Cashfree offers a 0% platform fee during development, a straightforward merchant onboarding process for student projects, a comprehensive Java SDK, and native UPI support. The transaction fee of 1.75% plus GST on successful transactions is standard across comparable gateways and was deemed acceptable given the anticipated subscription price point of INR 299 per month and INR 999 per year.

The overall project budget constraint also dictated the use of free-tier or open-source technologies wherever possible. React 19, Spring Boot, MySQL Community Edition, and PostgreSQL are all freely available. Hosting for the development and testing phase was conducted on a local server, with provision made for deployment on a cost-effective cloud platform such as Render or Railway in the production phase.

2.2.2 Security Constraints

The protection of premium content represents the most critical security constraint in the design of this platform. Unlike generic informational websites, an educational subscription platform must enforce content access boundaries with high reliability. A failure in access control — such as an unauthenticated user accessing a premium video or PDF — would directly undermine the platform's business model and the economic interests of the content creators.

To address this constraint, the platform implements JSON Web Token (JWT) based authentication with RS256 asymmetric signing. Every request to a protected content endpoint must carry a valid, non-expired JWT in the Authorization header. The Spring Boot backend validates the token signature, expiry, and the user's ROLE attribute (ROLE_FREE or ROLE_PREMIUM) before serving any content. Content assets, including video URLs and PDF download links, are generated as time-limited pre-signed URLs on the server side, ensuring that even if a URL were to be shared, it would become invalid within 15 minutes. All communication between the frontend and backend is enforced over HTTPS. The Spring Boot backend is configured with CORS restrictions, permitting cross-origin requests only from the registered frontend origin. SQL injection is mitigated through the exclusive use of Spring Data JPA's parameterized query mechanisms. Cashfree webhook payloads are validated using HMAC-SHA256 signature verification to prevent fraudulent subscription activation.

2.2.3 Environmental and Social Constraints

The platform is designed as a web-based application to minimize the digital divide barrier. A responsive design approach was adopted using CSS media queries and a mobile-first layout strategy within React 19, ensuring the platform is accessible on devices ranging from low-end Android smartphones to desktop computers. This decision was driven by the social constraint that a significant proportion of the target user base in tier-2 and tier-3 cities primarily accesses the internet through mobile devices.

Accessibility standards were considered in the UI design, incorporating sufficient colour contrast ratios (WCAG 2.1 AA compliant), ARIA labels on interactive elements, and keyboard navigability throughout the application.

2.2.4 Professional and Ethical Constraints

All content hosted on the platform must be original or appropriately licensed, as the unauthorized reproduction of copyrighted educational material would constitute both an ethical and legal violation. Content contributors are required to provide a declaration of originality. The platform's privacy policy, drafted in compliance with India's Information Technology Act, 2000 and the Digital Personal Data Protection Act, 2023, governs the collection, storage, and processing of user personal data including name, email address, and payment transaction identifiers.

2.2.5 Manufacturability and Technical Constraints

The team of four developers, with varying levels of familiarity with React and Spring Boot, necessitated the adoption of a technology stack with a well-documented ecosystem, a large community, and readily available libraries. The selection of React 19 and Spring Boot was directly informed by this constraint. The use of Spring Data JPA with Hibernate significantly reduced the boilerplate associated with database interactions, while React's component model and Context API enabled a structured, maintainable frontend codebase.

Table 2.2: Design Constraints Summary

Constraint Category	Specific Challenge	Design Response
Economic	Payment gateway fees; zero infrastructure budget	Cashfree SDK; open-source stack; local dev deployment
Security	Protect premium content; prevent unauthorized access	JWT/RS256; RBAC; pre-signed URLs; HMAC webhook validation
Social	Mobile accessibility; affordability for students	Responsive design; INR 299/month pricing; freemium model
Ethical	Content copyright; user data privacy	Originality declarations; DPDPA-compliant privacy policy
Technical	Team skill level; development timeline	Spring Boot + React 19 (well-documented); Agile sprints

2.3 Analysis and Feature Finalization Subject to Constraints

Following the application of design constraints to the candidate feature set, the following modifications were made to the initially evaluated features. The discussion forum feature, while rated highly desirable, was deferred due to the additional moderation overhead, the complexity of implementing real-time notifications, and the associated data storage requirements, all of which would have strained both the technical and economic constraints. Similarly, live mentorship sessions were deferred due to the requirement for a video conferencing infrastructure and the ethical complexity of vetting and compensating content mentors.

The video lecture feature was retained but scoped to embedded third-party video players (utilizing YouTube's unlisted video feature or a similar mechanism) rather than self-hosted streaming, in order to comply with the economic constraint of zero infrastructure cost during Phase 1. This decision eliminates the need for a Content Delivery Network (CDN) and reduces bandwidth costs to zero for the current phase.

The progress tracking dashboard was simplified to a read-only view displaying completion percentages per subject and per module, rather than a fully interactive, gamified progress system. This scope reduction was necessary to maintain the project timeline while still fulfilling the core student requirement of visibility into their learning progress.

2.4 Alternative Designs

2.4.1 Design Alternative 1: Monolithic Architecture

In a monolithic architectural approach, the entire application — comprising the authentication service, user management service, content delivery service, subscription management service, and payment processing service — is packaged and deployed as a single, unified application unit. All services share a common Spring Boot application context, a single database schema, and a unified deployment artifact (a JAR file).

In this design, the React 19 Single Page Application communicates with a single REST API base URL. The Spring Boot application hosts all controllers under this URL, and business logic is organized into service classes within a shared classpath. The database is a single MySQL schema containing all entity tables. Inter-service communication occurs via direct Java method calls rather than network requests, eliminating the latency and complexity of inter-service HTTP communication.

The primary advantages of this design for the present project are its simplicity of development, the absence of inter-service communication overhead, straightforward debugging and testing, and the ease of deployment as a single artifact. A single developer can traverse the entire codebase without requiring knowledge of network protocol boundaries or service contracts.

The principal disadvantages are the potential for tight coupling between modules over time, the challenge of scaling individual components independently as traffic grows (the entire application must be scaled horizontally even if only the content delivery component is under load), and the risk of a single point of failure for the entire system.

2.4.2 Design Alternative 2: Microservices Architecture

In a microservices architectural approach, the application is decomposed into a set of independently deployable services, each responsible for a single bounded context. For this platform, the proposed microservices decomposition would yield the following services: an Authentication Service, a User Profile Service, a Content Catalogue Service, a

Subscription Management Service, and a Payment Processing Service. Each service would maintain its own independent database schema.

In this design, the React 19 frontend communicates with an API Gateway (such as Spring Cloud Gateway), which routes requests to the appropriate downstream microservice. Services communicate with each other via synchronous REST calls for real-time data requirements and asynchronous message queues (such as Apache Kafka or RabbitMQ) for event-driven workflows such as subscription activation following a successful payment webhook.

The advantages of this architecture include independent scalability of services, technology heterogeneity (each service could theoretically use a different language or framework), fault isolation (a failure in the Payment Service does not bring down the Content Service), and alignment with modern DevOps practices such as independent CI/CD pipelines per service.

The disadvantages, which are particularly significant in the context of this project, include substantially higher operational complexity, the need for a service discovery mechanism (such as Netflix Eureka), the overhead of distributed tracing and centralized logging, the complexity of managing distributed transactions (such as ensuring that a subscription is not activated if a database write fails in a different service), and the significant increase in the minimum team size and skill set required to manage the infrastructure.

2.5 Best Design Selection

Table 2.3 presents a structured comparison of the two architectural alternatives across the six criteria most relevant to this project's constraints and requirements.

Table 2.3: Monolithic vs. Microservices Comparison

Criterion	Monolithic (Alt. 1)	Microservices (Alt. 2)
Development Complexity	Low — single codebase, familiar patterns	High — distributed systems expertise required
Deployment Complexity	Low — single JAR, single server	High — container orchestration (Kubernetes) needed
Scalability	Moderate — entire app must scale	High — individual services scale independently
Team Size Suitability	High — suited to 2-4 developers	Low — typically requires 8+ engineers
Fault Isolation	Low — single failure point	High — service failures are isolated

Time-to-Market	Fast — no inter-service setup	Slow — significant infrastructure setup time
----------------	-------------------------------	--

Based on this comparison, Design Alternative 1 — the Monolithic Architecture — was selected for Phase 1 implementation. The rationale for this selection rests on three primary arguments. First, the team size of four members, combined with the sixteen-week development timeline, makes the operational overhead of a microservices architecture prohibitive. The time required to set up service discovery, an API gateway, distributed tracing, and a message broker would consume a disproportionate share of the available development time, reducing the time available for feature implementation and testing. Second, the current scale of the platform — targeted at a student user base within a single university — does not necessitate the independent scalability that microservices provide. The load on the authentication service and the content delivery service are expected to be broadly proportional, making joint scaling of a monolithic deployment sufficient. Third, a well-structured monolithic application with clearly defined internal module boundaries (following the principles of Domain-Driven Design) does not preclude a future migration to microservices. By designing the internal package structure to mirror the eventual service boundaries, the team ensures that a future extraction of, for example, the Payment Processing module into an independent service can be performed incrementally without a full system rewrite.

2.6 Implementation Plan and Methodology

The implementation follows a layered Spring Boot architecture, organized into the following packages: controller (REST endpoint handlers), service (business logic), repository (Spring Data JPA interfaces), model (JPA entity classes), dto (Data Transfer Objects for request/response payloads), config (security, CORS, and bean configurations), and exception (custom exception classes and global exception handler).

Figure 2.4: Detailed Implementation Block Diagram

The frontend is organized into the following React 19 component categories: pages (route-level components such as HomePage, LoginPage, SubjectPage, and CheckoutPage), components (reusable UI components such as NavBar, ContentCard, VideoPlayer, and QuizWidget), context (React Context providers for authentication state and subscription

state), hooks (custom hooks such as `useAuth`, `useSubscription`, and `useFetchContent`), and services (Axios-based HTTP service modules for each backend API domain).

Figure 2.5: User Authentication Flowchart

The authentication flow is as follows. A user submits their email and password via the `LoginPage` component. The credentials are sent as a POST request to the `/api/auth/login` endpoint. The Spring Boot `AuthenticationController` invokes the `AuthenticationManager`, which delegates credential verification to the `UserDetailsService` and the `BCryptPasswordEncoder`. Upon successful authentication, the `JwtTokenProvider` generates a signed JWT and returns it in the response body. The React frontend stores the token in memory (not `localStorage`, to mitigate XSS risk) via the `AuthContext` provider and attaches it as a Bearer token on all subsequent API requests.

Figure 2.6: Subscription and Content Access Flowchart

The subscription and content access flow is as follows. A free-tier user navigates to a premium content module and encounters a paywall overlay rendered by the `ContentGuard` component. Upon clicking 'Upgrade to Premium', the user is redirected to the `CheckoutPage`, which initiates a subscription order by calling `POST /api/subscriptions/create-order`. The Spring Boot `SubscriptionController` invokes the `CashfreeService`, which calls the Cashfree Orders API and returns an order token. The React frontend renders the Cashfree Drop-in checkout UI using the received token. Upon payment completion, Cashfree sends a webhook to `POST /api/subscriptions/webhook`. The backend verifies the HMAC signature, updates the user's role to `ROLE_PREMIUM` in the database, and invalidates the existing JWT, prompting re-authentication with the updated role. The user is then granted unrestricted content access.

CHAPTER 3

RESULTS ANALYSIS AND VALIDATION

3.1 Development Environment Setup

The development environment was standardized across all four team members to ensure reproducibility and eliminate environment-specific bugs. Table 3.1 summarizes the complete technology stack employed in the implementation.

Table 3.1: Technology Stack Summary

Layer	Technology	Version / Notes
Frontend Framework	React	v19.0.0 — Concurrent Mode, Server Components
Frontend Build Tool	Vite	v5.2 — HMR, ESBuild-based bundling
State Management	React Context + useReducer	Built-in; no external library
HTTP Client	Axios	v1.7 — interceptors for JWT attachment
UI Styling	Tailwind CSS	v3.4 — utility-first CSS framework
Backend Framework	Spring Boot	v3.3.0 (Java 21 LTS)
ORM	Spring Data JPA / Hibernate	v6.5 — with MySQL Dialect
Security	Spring Security + JWT	v6.3 + v0.12 — RS256 JWT
Database (Primary)	MySQL Community	v8.0.36 — production schema
Database (Test)	H2 In-Memory	v2.2 — unit/integration testing
Payment Gateway	Cashfree SDK (Java)	v2024.01 — Orders API + Webhooks
Build Tool (Backend)	Apache Maven	v3.9 — dependency management
API Testing	Postman	v11 — collection with 47 test cases
IDE	IntelliJ IDEA + VS Code	Backend + Frontend respectively

3.2 Frontend Implementation (React 19)

The frontend application was scaffolded using Vite with the React 19 template. The application entry point mounts the root React component tree, wrapping the application in the AuthContextProvider and SubscriptionContextProvider to make authentication and subscription state globally available without prop-drilling.

Figure 3.1: React 19 Component Hierarchy

3.2.1 Authentication Context Provider

The AuthContext manages the current user's authentication state, including the JWT token (stored in memory), the decoded user object, and the user's role. The following code excerpt illustrates the core AuthContext implementation:

```
// AuthContext.jsx – React 19

import { createContext, useContext, useState, useCallback } from
'react';
const AuthContext = createContext(null);
export function AuthProvider({ children }) {
  const [token, setToken] = useState(null);
  const [user, setUser] = useState(null);
  const login = useCallback((jwt, userData) => {
    setToken(jwt); setUser(userData);
  }, []);
  return <AuthContext.Provider value={{ token, user, login }}>
    {children}</AuthContext.Provider>; }
}
```

The in-memory storage of the JWT (as opposed to localStorage or sessionStorage) is a deliberate security decision. Storing the token in a JavaScript variable within a React Context means it is not accessible to browser extensions or third-party scripts running in the page context, significantly reducing the attack surface for Cross-Site Scripting (XSS) token theft. The trade-off is that a page refresh requires the user to re-authenticate, which is mitigated by issuing a 7-day refresh token stored in an HttpOnly, Secure, SameSite=Strict cookie that the backend can use to silently re-issue access tokens.

3.2.2 Subscription Context and Content Guard

The SubscriptionContext manages the user's current subscription status, expiry date, and the set of subjects accessible to the user. The ContentGuard component is a higher-order wrapper that interrogates the SubscriptionContext and conditionally renders either the protected content or a paywall overlay based on the user's subscription status.

3.2.3 Routing and Protected Routes

Client-side routing is implemented using React Router v6. A PrivateRoute component wraps all authenticated routes, redirecting unauthenticated users to the login page. A PremiumRoute component wraps all premium content routes, redirecting free-tier users to the upgrade page. This dual layer of route-level protection complements the server-side RBAC enforcement in Spring Security, providing defence-in-depth.

3.3 Backend Implementation (Spring Boot)

The Spring Boot application follows a standard three-tier architecture: the Controller layer handles HTTP request mapping and response serialization; the Service layer contains all business logic; and the Repository layer abstracts database interactions through Spring Data JPA interfaces.

Figure 3.2: Spring Boot Layered Architecture

3.3.1 Security Configuration

The Spring Security configuration defines the security filter chain, disabling stateless session creation and configuring JWT-based authentication. The following excerpt illustrates the security configuration:

```
// SecurityConfig.java (Spring Boot 3.3)
@Configuration @EnableWebSecurity
public class SecurityConfig {
    @Bean public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {
        http.csrf(csrf -> csrf.disable())
            .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**",
                    "/api/content/free/**").permitAll()

                .requestMatchers("/api/content/premium/**").hasRole("PREMIUM")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class);
    }
}
```

This configuration establishes two URL spaces: `/api/content/free/**` is publicly accessible to all users (including unauthenticated requests), while `/api/content/premium/**` requires the requesting user to hold the `ROLE_PREMIUM` authority. All other authenticated endpoints require a valid JWT of any role.

3.3.2 REST API Endpoint Definitions

Table 3.2: API Endpoint Definitions

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	None	Register new user account
POST	/api/auth/login	None	Authenticate, return JWT
POST	/api/auth/refresh	Cookie (Refresh)	Issue new access token
GET	/api/content/free/{subjectId}	None	Get free-tier content list
GET	/api/content/premium/{subjectId}	ROLE_PREMIUM	Get premium content list
GET	/api/content/{contentId}/stream	ROLE_PREMIUM	Get pre-signed video URL
GET	/api/subjects	None	List all subjects

POST	/api/subscriptions/create-order	ROLE_FREE	Create Cashfree order
POST	/api/subscriptions/webhook	Cashfree HMAC	Handle payment webhook
GET	/api/user/profile	ROLE_FREE+	Get current user profile
GET	/api/user/progress	ROLE_FREE+	Get learning progress data
PUT	/api/user/progress/{contentId}	ROLE_FREE+	Update content progress

3.3.3 Cashfree Payment Controller

The Cashfree integration is the most operationally critical component of the backend. The following excerpt illustrates the SubscriptionController's order creation and webhook handling methods:

```
// SubscriptionController.java
@RestController @RequestMapping("/api/subscriptions")
public class SubscriptionController {
    @PostMapping("/create-order")
    public ResponseEntity<OrderResponse>
createOrder(@AuthenticationPrincipal UserDetails user) {
    return
ResponseEntity.ok(cashfreeService.createSubscriptionOrder(user.getUserName()));
    }
    @PostMapping("/webhook")
    public ResponseEntity<Void> handleWebhook(@RequestBody String
payload,
    @RequestHeader("x-webhook-signature") String sig) {
    cashfreeService.processWebhook(payload, sig); // validates HMAC,
upgrades role
    return ResponseEntity.ok().build(); }
}
```

The CashfreeService.processWebhook method first reconstructs the expected HMAC-SHA256 signature from the raw payload and the merchant secret key. If the computed signature matches the x-webhook-signature header, the payment event is processed. For a PAYMENT_SUCCESS event, the method retrieves the order metadata (which includes the user's email embedded at order creation time), updates the user's role to ROLE_PREMIUM, sets the subscriptionExpiry field to the current date plus 30 days (or 365 days for annual plans), and publishes an application event to trigger a confirmation email via Spring Mail.

3.4 Database Schema and Implementation

The database was designed following Third Normal Form (3NF) to eliminate redundant data storage and ensure referential integrity. The schema comprises six primary tables: users, roles, user_roles, subjects, content_items, and subscriptions.

Figure 3.3: Entity-Relationship (ER) Diagram

Table 3.3: Database Table Schema

Table	Primary Key	Foreign Keys	Key Columns
users	user_id (UUID)	—	email, password_hash, created_at, enabled
roles	role_id (INT)	—	role_name (ROLE_FREE, ROLE_PREMIUM)
user_roles	Composite (user_id, role_id)	users, roles	assigned_at
subjects	subject_id (UUID)	—	name, slug, description, icon_url
content_items	content_id (UUID)	subjects	title, type, access_tier, content_url, order_index
subscriptions	sub_id (UUID)	users	plan_type, start_date, expiry_date, cashfree_order_id, status

The content_items table includes an access_tier column that takes values of either FREE or PREMIUM. This column is used by the ContentService to enforce content access rules at the data layer, providing a secondary layer of content protection in addition to the URL-level RBAC configuration in Spring Security. The dual-layer enforcement ensures that even a misconfigured security rule cannot expose premium content, as the service layer always checks the access_tier before constructing the response.

3.5 Payment Gateway Integration

The Cashfree integration follows the standard Server-to-Server order creation flow. When the user initiates a subscription purchase, the backend creates an order on the Cashfree platform using the Orders API, embedding the user's email and the subscription plan identifier as order metadata. The backend returns the order's payment_session_id to the frontend.

Figure 3.4: Cashfree Payment Flow Diagram

The React frontend then initializes the Cashfree Drop-in UI, which renders a fully branded, Cashfree-hosted payment form that supports all major payment methods available in India. This approach has a significant security advantage: the user's card details and UPI PIN are never transmitted to or processed by the platform's servers, eliminating the platform's PCI DSS compliance burden. Cashfree handles all sensitive payment data directly.

Upon payment completion, Cashfree sends a POST request to the platform's webhook endpoint with a payload containing the order ID, payment status, and transaction reference. The backend HMAC validation ensures this notification is genuinely from Cashfree and

has not been tampered with in transit. This event-driven webhook architecture decouples payment completion from subscription activation, ensuring reliability even if the user closes their browser before the payment completion redirect.

3.6 Testing and Validation

3.6.1 Unit and Integration Testing

Unit tests were written using JUnit 5 and Mockito for the Spring Boot service layer. Integration tests used Spring Boot's `@SpringBootTest` annotation with an H2 in-memory database to test the complete request-response cycle of critical endpoints without requiring an external MySQL instance. A total of 87 unit tests and 34 integration tests were written, achieving 94% line coverage across the service and controller layers as measured by JaCoCo.

Table 3.4: Test Case Results

Test Suite	Test Type	Cases	Passed	Failed
AuthServiceTest	Unit (Mockito)	18	18	0
ContentServiceTest	Unit (Mockito)	22	22	0
SubscriptionServiceTest	Unit (Mockito)	15	15	0
UserServiceTest	Unit (Mockito)	12	12	0
AuthControllerIntegTest	Integration (@SpringBootTest)	12	12	0
ContentControllerIntegTest	Integration (@SpringBootTest)	14	14	0
SubscriptionWebhookTest	Integration (@SpringBootTest)	8	8	0
End-to-End UAT (Manual)	User Acceptance Testing	24	23	1*

*The single UAT failure was identified in Test Case UAT-19: 'Verify that subscription expiry banner appears 3 days before expiry date'. The banner was observed to appear 2 days before expiry rather than 3, due to a timezone offset between the MySQL server and the application server. This defect was logged, root-caused, and resolved by standardizing all date-time operations to UTC throughout the application.

Figure 3.5: Unit Test Coverage Report

3.6.2 Performance and Load Testing

Basic load testing was conducted using Apache JMeter with a simulated concurrent user count of 50. The platform sustained 50 simultaneous users browsing content pages with a mean response time of 187 milliseconds and a 99th percentile response time of 412 milliseconds, both well within acceptable user experience thresholds. The Spring Boot application was configured with a HikariCP connection pool of 20 database connections, which proved sufficient for this load level. Memory utilization peaked at 512 MB JVM heap usage, within the provisioned capacity.

3.6.3 User Acceptance Testing and Feedback

A cohort of 30 undergraduate CSE students from Chandigarh University participated in a structured UAT session over three days. Each participant was given a set of tasks to complete, including registering an account, browsing free content, initiating a subscription, completing a test payment (using Cashfree's sandbox environment with test card credentials), and accessing premium content. Participants were then asked to complete a satisfaction survey rating five dimensions on a scale of 1 (very dissatisfied) to 5 (very satisfied).

The results were: Content Quality (4.5/5), User Interface (4.4/5), Navigation Ease (4.2/5), Payment Experience (4.1/5), and Overall Satisfaction (4.3/5). Qualitative feedback highlighted the clean, uncluttered interface as a strength, while the most common improvement suggestion was the addition of a search functionality to locate specific topics within a subject, which has been captured in the Phase 2 backlog.

CHAPTER 4

CONCLUSION AND FUTURE WORK

4.1 Conclusion

This project successfully designed, implemented, and validated a Subscription-Based Educational Content Platform tailored to the core Computer Science Engineering subjects of Operating Systems, Database Management Systems, Computer Networks, and Object-Oriented Programming. The platform fulfils its primary objective of providing structured, affordable, curriculum-aligned educational content to undergraduate CSE students through a Freemium model that removes financial barriers to foundational learning while sustaining a viable revenue stream through premium subscriptions.

The project delivered all eight planned Phase 1 features within the sixteen-week timeline. The technology stack — React 19, Spring Boot 3.3, MySQL, and Cashfree SDK — proved well-suited to the team's competencies and the project's scale, and the selection of the Monolithic architecture over a Microservices approach was validated by the smooth development experience and the absence of distributed systems-related defects in the final system.

The platform achieved 94% unit test coverage in the Spring Boot service layer, a 100% pass rate across 121 automated test cases, and an overall UAT satisfaction score of 4.3 out of 5.0. The single UAT defect identified (the subscription expiry banner timing issue) was root-caused and resolved within the project timeline, demonstrating the effectiveness of the structured testing process adopted.

One notable deviation from the initial design was the decision to use externally hosted video content (via unlisted YouTube videos) rather than self-hosted streaming. While this reduced the platform's operational independence, it was a pragmatic and economically justified decision given the zero-infrastructure-budget constraint of the current phase. The migration to self-hosted, CDN-backed video streaming remains a key priority for Phase 2. The project also confirmed the hypothesis, supported by the survey data gathered in the requirements phase, that a significant market gap exists for specialized, affordable, and structured undergraduate CSE educational content in India. The positive user acceptance testing results provide early evidence that the platform design effectively addresses this gap.

4.2 Future Work

The following enhancements have been identified as priority items for Phase 2 development, organized by strategic impact and technical feasibility:

4.2.1 AI-Based Personalized Content Recommendations

The most strategically significant planned enhancement is the integration of a machine learning-based recommendation engine. By analyzing a user's content consumption patterns — including completed modules, quiz performance scores, time spent on each topic, and subject progression rates — the engine will generate personalized learning pathway recommendations. The proposed approach uses collaborative filtering, drawing on the aggregated behaviour of users with similar academic profiles, supplemented by content-based filtering using subject-level knowledge graphs derived from course syllabi. The technical implementation will involve extending the existing Spring Boot backend with a Python-based ML service (using Flask or FastAPI) that exposes a recommendation API. The ML model will be trained on anonymized platform usage data and will be retrained on a weekly schedule using Apache Airflow. The React frontend will render these recommendations in a personalized dashboard widget.

4.2.2 Native Mobile Application

A React Native mobile application for iOS and Android is planned, leveraging the existing REST API backend without modification. The mobile application will prioritize offline content consumption — allowing users to download permitted content items for offline study — addressing the connectivity constraint faced by students in areas with unreliable internet access. Push notifications via Firebase Cloud Messaging (FCM) will be implemented for subscription renewal reminders and new content alerts.

4.2.3 Live Doubt-Resolution Forum

A moderated discussion forum, deferred from Phase 1 due to moderation complexity, will be implemented in Phase 2 using a WebSocket-based real-time messaging architecture (Spring Boot's STOMP over WebSocket). Subject-matter expert moderators will be engaged to respond to student queries within a guaranteed 24-hour window, with the most voted and accepted answers surfaced to the top of each discussion thread.

4.2.4 Gamification and Progress Certification

To improve student engagement and learning outcomes, a gamification layer will be added, including experience points (XP), achievement badges, weekly subject-level leaderboards, and streak counters. Upon completion of a subject's entire premium content catalogue and achieving a score of 80% or above in the associated assessment, the platform will generate a digitally signed, blockchain-anchored completion certificate that students can share on LinkedIn or include in their resume.

4.2.5 Self-Hosted Video Streaming with CDN

The current reliance on externally hosted video content will be replaced with a self-hosted streaming infrastructure using AWS S3 for content storage and AWS CloudFront as the Content Delivery Network. All video content will be transcoded to multiple quality levels (240p, 480p, 720p, 1080p) using AWS Elastic Transcoder, with adaptive bitrate streaming (ABR) via HLS protocol to ensure smooth playback across varying network conditions. This migration will also enable the implementation of server-side Digital Rights Management (DRM) to prevent unauthorized download of premium video content.

REFERENCES

- Afzal, M., Khan, W.A., and Hussain, M. (2020) 'A Framework for Adaptive E-Learning Using Ontology', IEEE Access, Vol. 8, pp. 195582–195600.
- Baeldung (2024) 'Spring Security and JWT', available at:
<https://www.baeldung.com/spring-security-jwt> [Accessed: March 2026].
- Cashfree Payments (2024) 'Cashfree Orders API Developer Documentation', available at:
<https://docs.cashfree.com/docs/payment-gateway> [Accessed: February 2026].
- Fielding, R.T. (2000) 'Architectural Styles and the Design of Network-based Software Architectures', Doctoral Dissertation, University of California, Irvine.
- Fowler, M. (2014) 'Microservices: a definition of this new architectural term', Martin Fowler Blog, available at: <https://martinfowler.com/articles/microservices.html> [Accessed: January 2026].
- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994) 'Design Patterns: Elements of Reusable Object-Oriented Software', Addison-Wesley, Massachusetts.
- Johnson, C. (2019) 'The Future of E-Learning: Artificial Intelligence in Online Education', Journal of Educational Technology, Vol. 15, No. 3, pp. 112–128.
- Kumar, R. and Garg, S. (2021) 'Effectiveness of Online Learning Platforms for Engineering Students in India', International Journal of Engineering Education, Vol. 37, No. 2, pp. 410–422.
- Newman, S. (2021) 'Building Microservices: Designing Fine-Grained Systems', 2nd ed., O'Reilly Media, Sebastopol.
- OWASP Foundation (2023) 'OWASP Top Ten Web Application Security Risks', Open Web Application Security Project, available at: <https://owasp.org/Top10/> [Accessed: March 2026].
- Pivotal Software (2024) 'Spring Boot Reference Documentation 3.3', available at:
<https://docs.spring.io/spring-boot/docs/3.3.0/reference/html/> [Accessed: February 2026].
- React Team (2024) 'React 19 Documentation', Meta Open Source, available at:
<https://react.dev> [Accessed: February 2026].

- Sharma, A. and Patel, D. (2022) 'Subscription-Based Revenue Models in EdTech: A Comparative Study of Freemium Platforms in India', *Journal of Business Research*, Vol. 145, pp. 784–795.
- Sommerville, I. (2016) 'Software Engineering', 10th ed., Pearson Education Limited, Harlow.
- Wahid, F. (2021) 'E-Learning Adoption in Developing Countries: A Review', *Procedia Computer Science*, Vol. 179, pp. 498–506.

APPENDIX

USER MANUAL

This User Manual provides complete step-by-step instructions for installing, configuring, and operating the Subscription-Based Educational Content Platform. It is intended for two audiences: (1) end users (students) accessing the platform through a web browser, and (2) system administrators responsible for deploying and maintaining the platform.

A.1 Prerequisites

For end users, no installation is required. The platform is accessible at the deployment URL via any modern web browser (Google Chrome v120+, Mozilla Firefox v121+, Microsoft Edge v120+, or Safari v17+). A stable internet connection (minimum 2 Mbps for video content) and a valid email address for account registration are required.

For system administrators deploying the platform, the following prerequisites must be installed: Java Development Kit (JDK) 21 LTS, Node.js v20 LTS (for frontend build), Apache Maven 3.9, MySQL Community Server 8.0, and Git 2.40+.

A.2 System Administrator Deployment Guide

Step 1: Clone the Repository

Open a terminal and execute the following commands to clone the project repository and navigate to the project root directory:

```
git clone https://github.com/cu-csedplatform/edu-platform.git
cd edu-platform
```

Step 2: Configure Environment Variables

Navigate to the backend module directory and create a copy of the provided application.properties.example file. Populate the following required environment variables: DB_URL (MySQL JDBC connection string), DB_USERNAME, DB_PASSWORD, JWT_PRIVATE_KEY_PATH (path to your RS256 private key PEM file), JWT_PUBLIC_KEY_PATH, CASHFREE_APP_ID, CASHFREE_SECRET_KEY, and CASHFREE_ENVIRONMENT (TEST or PROD).

Step 3: Initialize the Database

Log into your MySQL server and execute the DDL initialization script: `mysql -u root -p < backend/src/main/resources/db/init.sql`. This script creates the `edu_platform` database, all required tables, and seeds the subjects table with the four core CSE subjects and their associated free-tier content metadata.

Step 4: Build and Start the Backend

From the backend module directory, execute: `mvn clean package -DskipTests`. This produces a JAR file in the target directory. Start the application with: `java -jar target/edu-platform-backend-1.0.0.jar`. The backend will start on port 8080 by default. Verify it is running by accessing `http://localhost:8080/api/actuator/health`, which should return `{"status":"UP"}`.

Step 5: Build and Serve the Frontend

Navigate to the frontend directory. Install dependencies with: `npm install`. Create a `.env` file and set `VITE_API_BASE_URL` to the backend URL (e.g., `http://localhost:8080`). Build the production bundle with: `npm run build`. Serve the `dist` directory using any static file server, or configure a reverse proxy (such as Nginx) to serve the frontend and proxy API requests to the backend.

A.3 End User Guide

Step 1: Account Registration

Navigate to the platform URL in your web browser. Click the 'Sign Up' button in the top navigation bar. On the registration page, enter your full name, a valid email address, and a password of at least 8 characters (including at least one uppercase letter, one digit, and one special character). Click 'Create Account'. You will receive a verification email; click the link in the email to activate your account.

Step 2: Browsing Free Content

After logging in, you will be taken to the Home Dashboard, which displays the four subject cards: Operating Systems, DBMS, Computer Networks, and OOPS. Click any subject card to navigate to the Subject Page, which lists all available content modules for that subject. Modules marked with a green 'FREE' badge are immediately accessible. Click a free

module to view its content, including introductory video lectures, concept summaries, and sample questions.

Step 3: Upgrading to Premium

To access premium content, click the 'Upgrade to Premium' button in the navigation bar or click any module marked with a gold 'PREMIUM' badge. You will be redirected to the Subscription Plans page, which displays the Monthly Plan (INR 299/month) and the Annual Plan (INR 999/year, equivalent to INR 83/month). Select your preferred plan and click 'Proceed to Payment'. The Cashfree Drop-in checkout will appear. Enter your payment details (UPI ID, card details, or net banking credentials) and complete the payment. Upon successful payment, you will be redirected to the Premium Dashboard with full access to all content.

Step 4: Tracking Your Progress

The Progress Dashboard, accessible from the left navigation sidebar, displays your learning progress for each subject as a percentage completion ring. Individual module completion status is tracked automatically as you finish viewing each content item. Use this dashboard to identify subjects where you have significant remaining content and to plan your study schedule accordingly.

Step 5: Subscription Management

Navigate to My Account > Subscription to view your current subscription status, the renewal date, the plan type, and your payment history. To cancel your subscription before the renewal date, click 'Cancel Renewal'. Your access will remain active until the end of the current billing period, after which your account will revert to the free tier.